

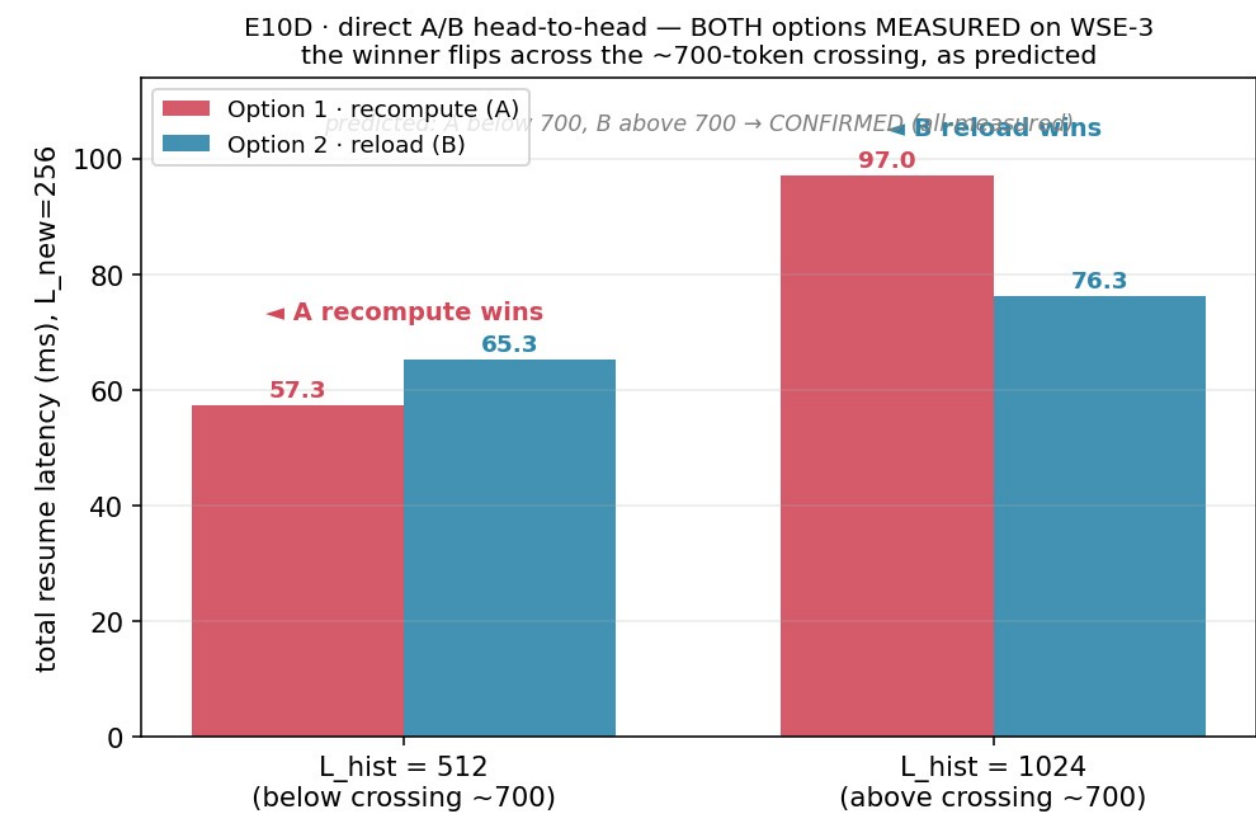
WEEKLY COLLABORATOR DISCUSSION

On-Chip KV Offload as a Storage Tier

Two multi-row storage designs measured on real WSE-3; the on-chip tier now sits under both prior resume options.

MODEL	EVIDENCE	SCOPE	DATE
Qwen3-1.7B geometry	81 CS-3 runs, 0-cycle spread	M3 idle-PE tier	2026-08-24

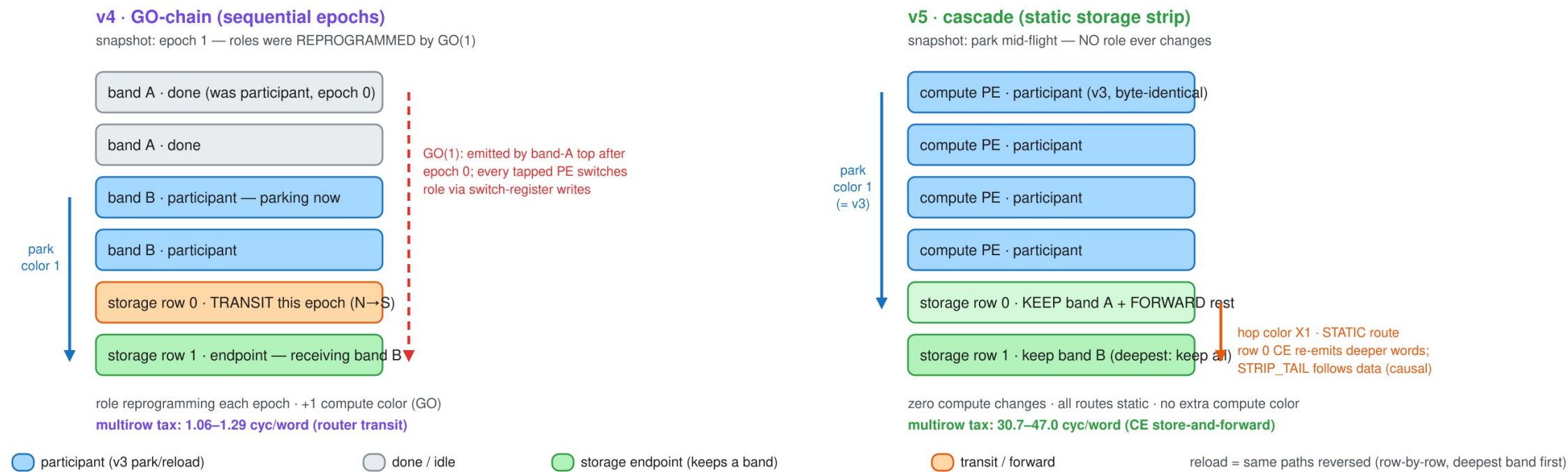
M2 ended at recompute-vs-host with a 744-token crossing; M3 asks if idle PEs beat both



Last time (M2 E10/E10D, all device-measured): below $H^*=744$ recompute wins, above it host reload wins — but reload rides shared edge IO and the host. M3's question: can idle on-chip PEs hold parked KV as a middle tier? This week: build the multi-row storage strip both candidate ways, measure both end-to-end, and price the tier.

v4 reprograms compute roles per epoch; v5 keeps compute byte-identical and cascades inside the strip

Two multi-row designs (R=2 shown) — snapshot: parking the second half of the column



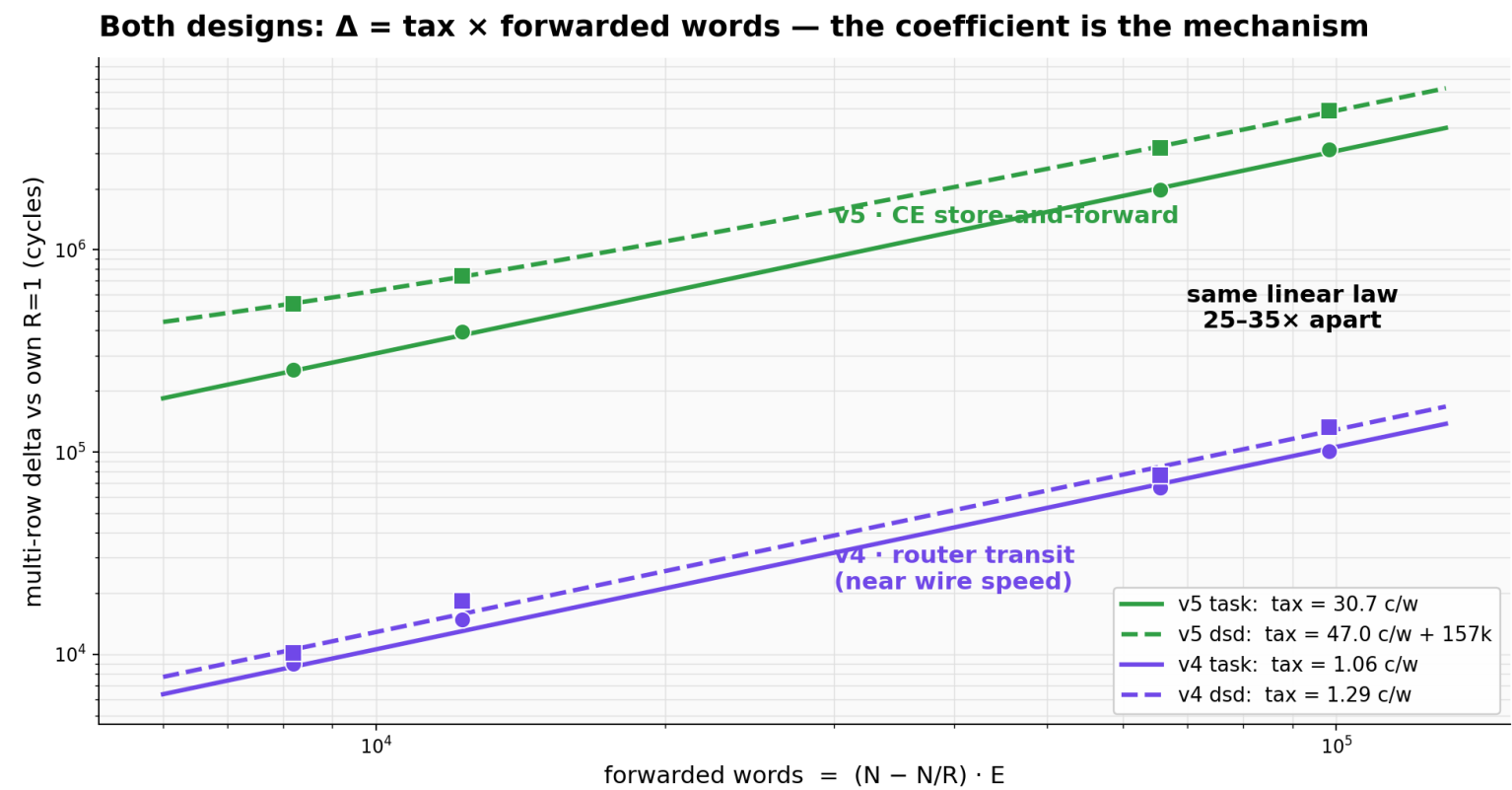
Both share the proven single-row protocol on one borrowed color. v4: bands park in sequential epochs, chained fully on-chip by a GO color; every tapped PE rewrites its switch positions per epoch. v5: row 0 keeps the nearest band and store-and-forwards deeper words on static per-hop colors; a STRIP_TAIL word rides behind the data as the causal end-of-stream. Reload returns over the same paths reversed.

CAPACITY · NOMINAL 42 KIB/STORAGE PE = 672 KV-TOKEN EQUIVALENTS PER 1×256 ROW/BLOCK

Each row adds 672-token capacity; v4 wins every multi-row setting

KV MOVED	ROWS · CAPACITY	V3 SINGLE-ROW	V4 GO-CHAIN	V5 CASCADE
task · 1,024 tok	1 · 672 tok (under)	1,388,470	1,535,920	1,420,244
task · 1,024 tok	2 · 1,344 tok (fits)	—	1,544,946	1,675,327
task · 1,024 tok	4 · 2,688 tok (fits)	—	1,550,855	1,812,526
DSD · 8,192 tok	1 · 672 tok (under)	7,350,234	8,542,434	7,482,812
DSD · 8,192 tok	2 · 1,344 tok (under)	—	8,619,349	10,669,498
DSD · 8,192 tok	4 · 2,688 tok (under)	—	8,675,026	12,343,610

Both designs obey $\Delta = \text{tax} \times \text{forwarded words}$ — the tax is the mechanism, 25-35x apart



Delta vs each design's own $R=1$ baseline collapses on the same axis, $\text{fwd_words} = (N - N/R) \cdot E$. v4's tax is router-level transit (1.06/1.29 cyc/word, near wire speed); v5's is a CE data-task per forwarded word (30.7/47.0). The v5 dsd fit needs one extra constant ($\sim 157\text{k}$ cycles): strip rows emit their own block before relaying, which fast owners expose.

Three device-fit models, one coefficient table

Fitted models (cycles; valid E ≥ 64 — E=4 sits in the floor regime; fwd = (N – N/R)·E)

v3

$$t = (245.4 \cdot N + 326) + (E-4) \cdot N \cdot (86.3 \mid 56.0) + 2(D-1) \cdot 2.0$$

v4

$$t = (61,058 \mid 78,676) + (E-4) \cdot N \cdot (96.00 \mid 65.08) + (1.06 \mid 1.29) \cdot \text{fwd}$$

v5

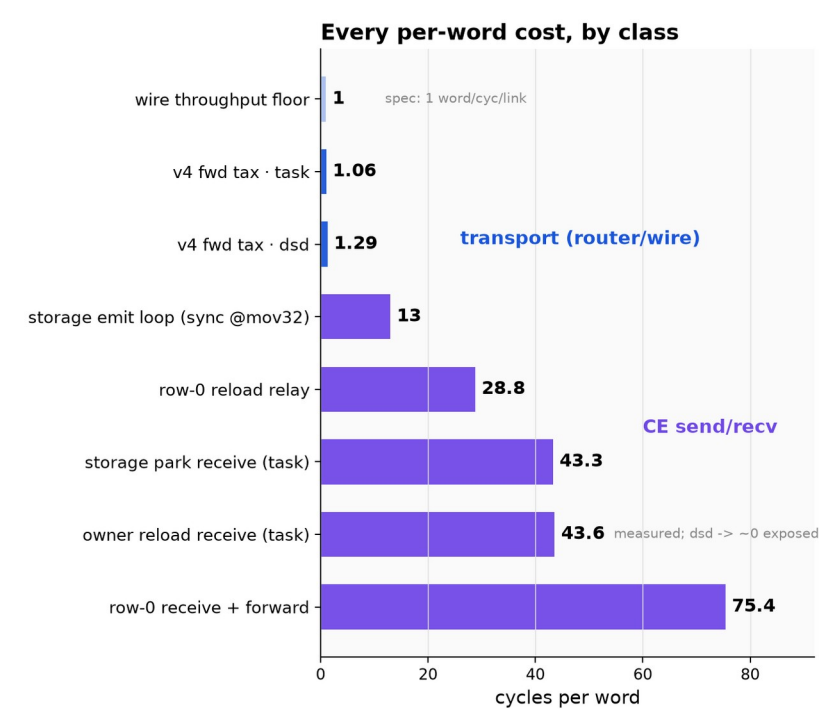
$$t = (70,561 \mid 70,076) + (E-4) \cdot N \cdot (87.87 \mid 57.00) + (30.7 \mid 47.0) \cdot \text{fwd} \quad [+157k \text{ dsd}, R>1]$$

(pairs are task | dsd · floors fitted at N=256 · selftest: 39 device anchors, v3 0.89% / v5 0.83% / v4 0.23%)

coefficient	value (task dsd)	what it measures · how it was fit
floor (per cycle)	245.4·N + 326	one-time work: per-row door-open flush chain + GO inject / join / fence landing fit: Exp-A E=4 cells, linear in N, max residual 8.8 ns
marginal_v3	86.3 56.0	full-cycle cost per (word,PE): park 43.3 + reload share (owner task 43.0 → dsd emit 13.0) fit: payload sweep + Exp-A (∝N to 0.03%); dsd split by Exp-B
marginal_v4	96.00 65.08	v3 + ~9.6 cyc/word: always-on role machinery (GO taps, switch writes) — paid even at R=1 fit: 2-point solve on v4's own R=1 cells (E=64, 512)
marginal_v5	87.87 57.00	v3 + ~1.6 cyc/word: cascade per-word branches that do not compile out at R=1 (H1 falsified) fit: 2-point solve on v5's own R=1 cells
tax_v4 (/fwd word)	1.06 1.29	deeper bands transit shallower rows at ROUTER level — no CE ever touches the word fit: least squares over 8 measured deltas, residual ≤ 0.25%
tax_v5 (/fwd word)	30.7 47.0	row-0 CE store-and-forwards every deeper word — a full data-task dispatch per word fit: Δ-vs-fwd slopes across R∈{2,4} × E∈{64,512}
c_fwd_word	75.4 ± 0.9	row-0 park receive+validate+re-emit; vs 43.3 receive-only ⇒ forwarding is a second task, not an increment fit: park-span decomposition, 4 independent cells agree
c_relay_word	≈ 28.8	row-0 reload relay per word (the bare synchronous emit loop alone is 13.0) fit: tax_dsd identity; task-mode cross-check agrees to 1.6%
dsd_strip_fill	≈ 157k cyc	v5 dsd R>1 constant: strip rows emit their own block before relaying — fast owners expose it fit: Δ-fit intercepts (153k / 162k)
t_hop_router	2.0 cyc/hop	pure fabric transit per extra row of distance; no per-word term ⇒ placement is latency-free fit: Exp-C, D ∈ {8, 32}

Internal-discussion reference: the three fitted equations (task | dsd pairs) and every coefficient with its meaning and fit provenance. All values are real CS-3 measurements; selftest reproduces 39 device anchors under 1%.

Pure transport is 1-2 cyc/word; CE send/recv is 13-75 — and each class has its own lever



TRANSPORT — knob: storage-PE placement

measured: $t_{hop} = 2.0$ cyc/hop, NO per-word term (Exp-C, D up to 32)
⇒ strip placement is latency-free; depth budget is capacity/routing
unmeasured: link sharing with live decode, multi-lane aggregation
(all runs so far are isolated single-lane)

CE — knob 1: thread model (task → DSD)

owner receive: 43.6 → ~0 exposed (Exp-B, measured)
storage park receive 43.3 and emit 13.0 are still task/sync-loop:
the next rung; projected ~4-5x on the tier marginal (projection)

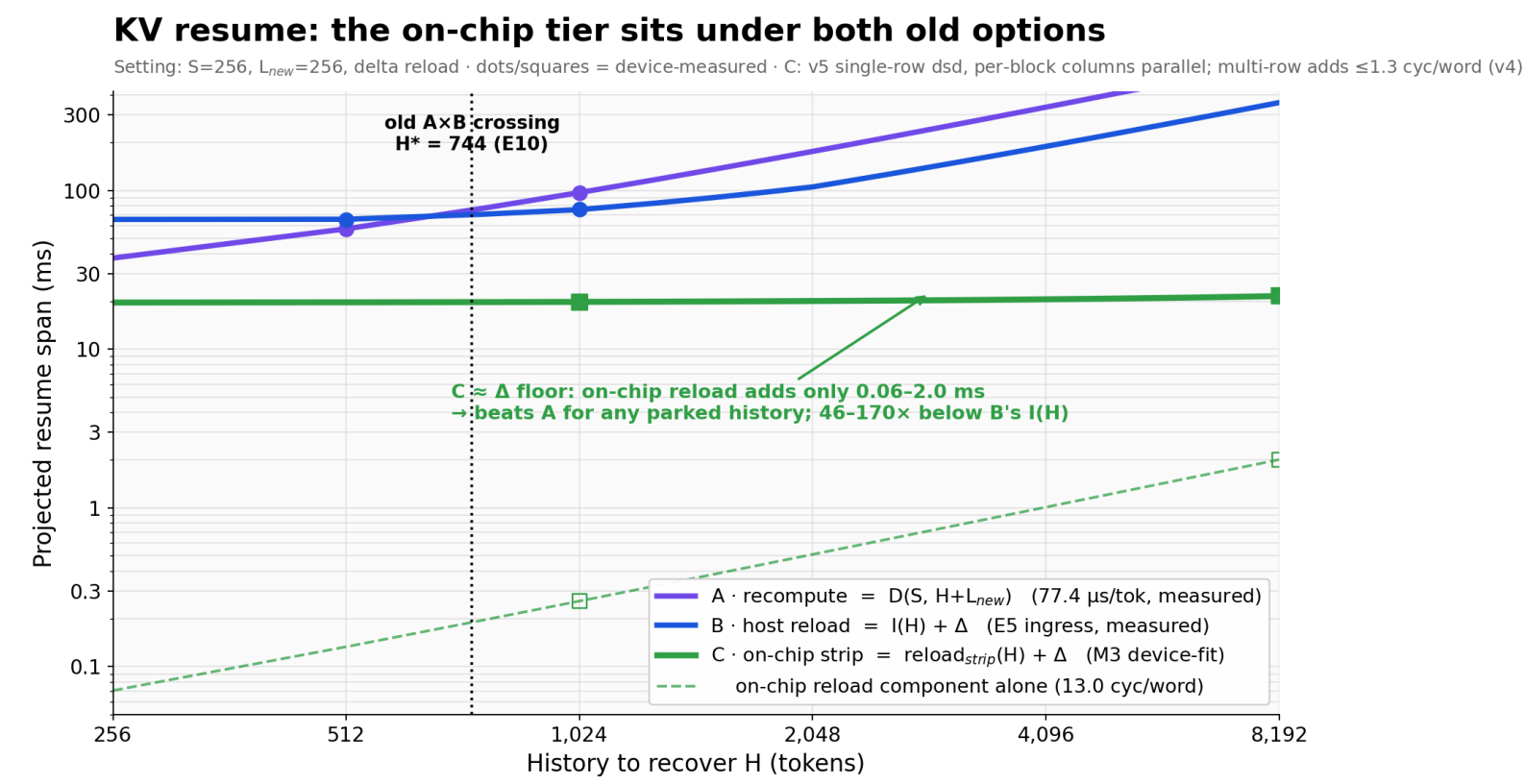
CE — knob 2: allocation across CEs

v5 concentrates: row-0 CE handles ALL N-E words (44-75 cyc each)
while deeper rows' CEs idle — tax 30.7-47.0
v4 distributes: each row's CE touches only its own band (bh-E);
inter-row movement is routers — tax 1.06-1.29
⇒ the 25-35x tax gap IS the allocation decision, measured

Left: every measured per-word cost, colored by class — transport (router/wire) vs CE send/recv. Right: what moves each class. Transport depends on storage-PE placement, measured latency-free (2.0 cyc/hop, no per-word term). CE depends on the thread model (task → DSD, measured on the owner side) and on how work is allocated across CEs: v5 concentrates all forwarding on row-0's CE (tax 30.7-47.0) while v4 leaves inter-row movement to routers (tax 1.06-1.29) — the 25-35x gap is the allocation decision itself, measured.

SO WHAT · THE TIER HOLDS

The on-chip tier sits under both old options: resume is now delta-dominated



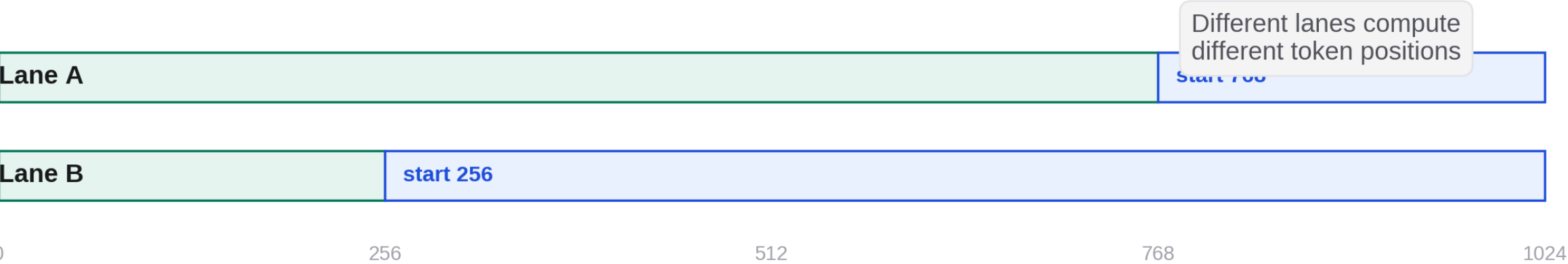
Same axes and setting as the M2 boundary figure, plus measured tier C: on-chip strip reload adds only 0.06-2.0 ms (13.0 cyc/word), 46-170x below host ingress $I(H)$, flattening resume onto the ~ 20 ms L_{new} delta floor. The old 744-token crossing becomes a fallback choice for un-parked history. Next lever for resume latency is the delta/restart path, not the transfer.

Common-start is correct—but recomputes longer prefix hits

Current: one common start for the whole batch

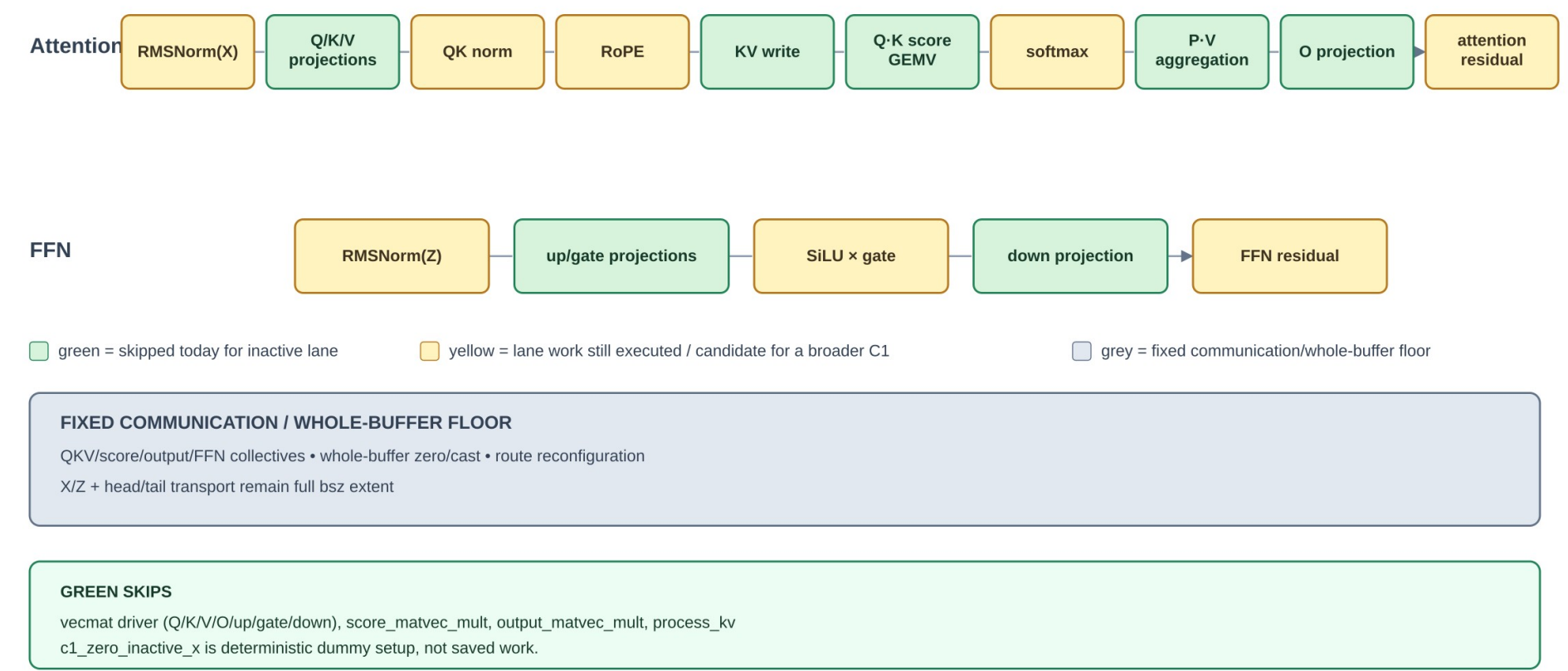


Needed: per-lane cursors inside the same round



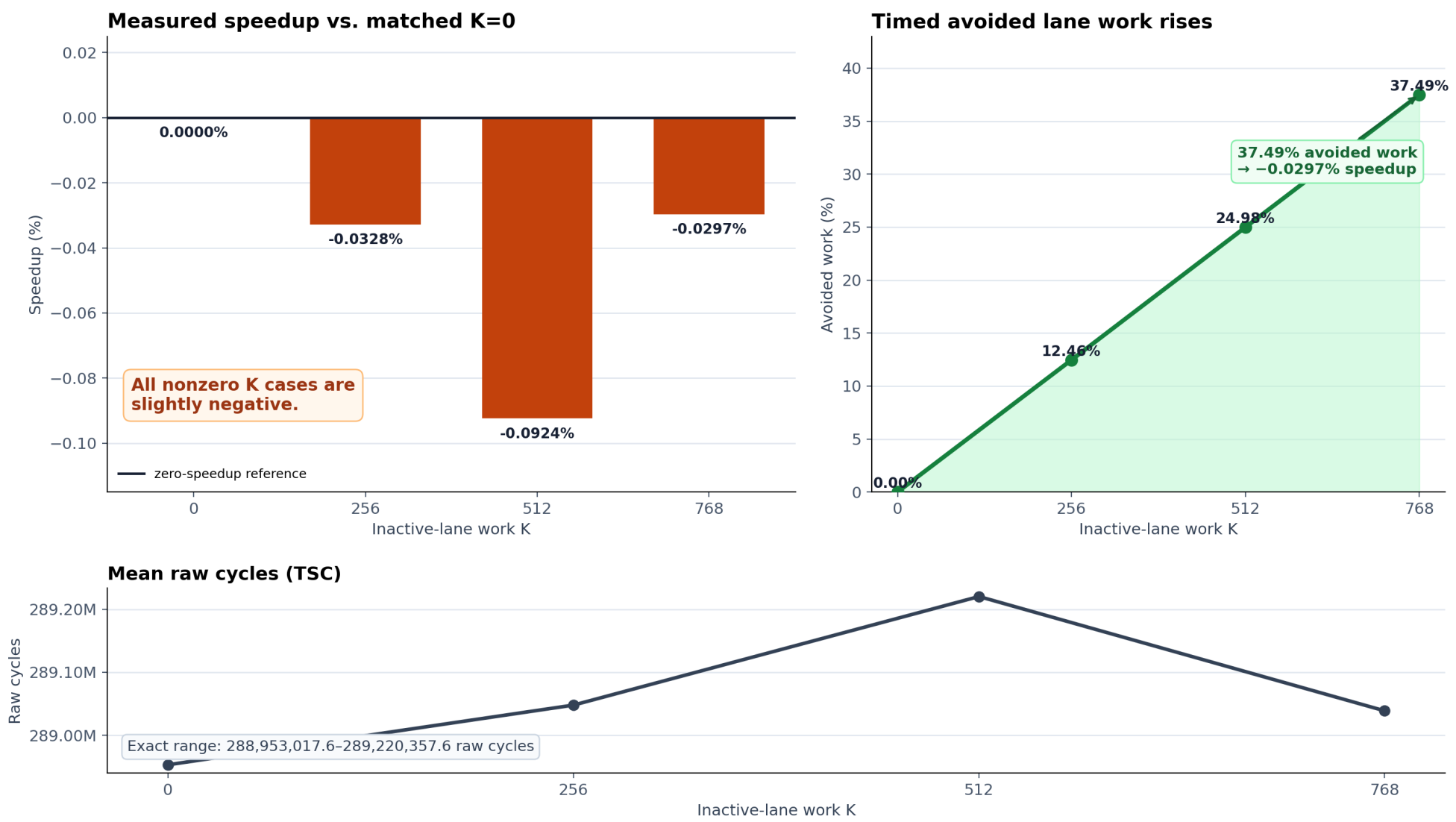
Lane A can reuse 768 tokens while Lane B can reuse only 256. The current batch starts both at 256, so Lane A re-executes 512 already-known token positions.

We skipped lane-sliced compute; the fixed batch floor stayed



Green is predicated while $\text{step} < R_i - \min(R)$; yellow still executes for the inactive lane; grey remains full-bsz communication or whole-buffer work.

37.49% less lane-step work produced no critical-path speedup



Full Qwen3-1.7B, bsz=2, S/N/F/G=256/1024/769/255, five repetitions per case, raw TSC cycles. At K=768, mean speedup is -0.0297%.

Pin the fixed floor—or move to independent lane progress

A · Profile the C1 fixed floor

- Time or ablate collectives, whole-buffer ops and transport by phase
- Identify the single largest remaining critical-path item
- Broaden masking only if that removable term can change the $<0.1\%$ result

B · Design C2 / continuous batching

- Independent per-lane start, cursor and completion state
- Price added SRAM, protocol, admission and scheduler complexity
- Only credit gains that survive a serving-like schedule replay